

eVTOL Simulation Problem

Design Choices

I started with a simple init, step, destroy set of functions in a simulation class. Added data structures for the various parameters in types.h. Took a state machine approach to the various vehicle states, airborne, ground, and charging. For the chargers, I added a queue / line / FIFO to allow for the vehicles to maintain their sequencing. This made the ground state a bit more involved than the other states, so I split the code into further functions. I used code for the queue I've written before, and I also used code for reading CSV files that I've written before. (but haven't put into any source control management online before, which I wanted to do anyways.)

In terms of libraries, I tend to favor stdio printf functions over C++ streams iostream. (setting various formatting options with iomanip has never struck me as elegant) although, now with C++20 `std::format()` provides something similar to Python f-strings, which seems less cumbersome.

I did the same with file operations, sticking with C library style versus C++ streams, although opening files with C++ streams isn't horrible.

The only allocation occurs in init, and the only deallocations occur within destroy, the csv file is opened allocated, parsed, and deallocated in main. I was tempted to support a dynamic amount of rows in the csv files, but that would require even more dynamic allocation, so I kept it statically defined.

I stuck with C++ new / delete for memory allocation and stuck with the default exception handler of exit the program if an allocation failure occurs. If I were using malloc and free I would check for NULL, then use perror to print error information with the errno value, and then return failure codes back to main, which usually would then exit anyway.

I kept things single threaded, although I would probably use OpenMP style parallel for operations to allow the main loop to simulate each vehicle in parallel. The issue really being with the charging stations causing a lot of contention. Obviously we can use semaphores or mutex variables to synchronize the threads. But then I think without careful planning we would lose any speed improvement and simply have more threads waiting around for resources. I've seen false sharing occur before even in cases that were perfectly parallel, so having results written to common memory can cause slow downs that are not obvious due to shared cache lines.

For unit testing I used google test, which works easily enough in a Linux environment. Since I started writing code in Visual Studio, it was at this point where most everything was working that I went ahead and got it running in an AWS instance, added it to git, and added makefiles, and got the basic unit testing framework running. I decided to use Doxygen style function and file headers also, so at this point I added those to the code base. (I'm not a huge fan of writing function and file headers, so I leveraged ChatGPT to write the headers, and it did a pretty good job)

In terms of function headers I have not been a huge fan of syncing file headers between .h and .cpp files, also I feel like function headers obscure the functions when you just want a high level overview of a class. So I opted to just add function headers to the .cpp files, and only use file headers in the .h files.

Since the code runs off the CSV files, I can use different input files to test using the same test runner, where typically I would have a unit test for a specific function or class in isolation. My main concern regarding testing is really that the statistics generated at the end are accurate. So I may be able to test this with simply different input files and validate that the outputs make sense. For example, set a cruise speed of 100 mph, a battery capacity of 100 kWh, and a charge time of one hour, an energy usage of one kWh/mile in order to keep all the numbers “round” as well as use the same stats for all vehicles so we are only focusing on a single use case.

This problem is a bit open ended, so one could easily spend a lot of time working on things. That said this week I was a bit busy, wrote most of the code on Tuesday afternoon, got it on github Wednesday afternoon and left it to sit until the weekend to allow for additional unit tests and writing this documentation due to other obligations. I think it would be nice to have latitudes and longitudes, a visualization of some sort (google earth, cesium, or similar) and perhaps even some gravity, thrust and gyro stabilization. But I figure keep it relatively bare bones is sufficient for our purposes.

